

Backend Guide

Automated Public Procurement Anomaly Detection • Easy explanation for the hackathon team

1. What is the backend?

The backend is the **brain of our project**. The frontend will show dashboards and graphs, but the backend will receive procurement files, clean the data, analyze it, find unusual patterns, calculate an investigation score, and send the results to the frontend.

Simple flow:

Excel / CSV → Python reads data → Clean data → Analyze patterns → Investigation Engine → Alerts + Score + Reasons → Dashboard

2. Why are we using Python?

Our team already knows Python, and Python is especially useful for this project because a large part of the work is **data processing and analysis**. We do not need to learn a completely new backend language.

3. Why pandas?

pandas is a Python library made for working with tables of data. Think of it as **Excel inside Python**. It can read CSV and Excel files, filter rows, calculate statistics, compare values, group records and prepare data for analysis.

Example: Suppose the uploaded file contains 50,000 bids. pandas can quickly load it and let us ask questions such as:

- How many tenders did each vendor participate in?
- How many did each vendor win?
- What was the average/median price for a category?
- Which bids are unusually close to each other?
- Which vendors repeatedly appear together?

CSV / Excel → pandas DataFrame → Analysis

4. The Investigation Engine

The Investigation Engine is the **main decision-support part** of our backend. It looks for different signals in the procurement data. A signal is simply a pattern that deserves a closer look.

We can start with 5 simple signals:

1. **Repeated Winner:** A vendor wins an unusually large percentage of similar tenders.
2. **Price Anomaly:** A contract price is much higher or lower than comparable procurements.
3. **Bid Similarity:** Multiple vendors repeatedly submit very similar bids.
4. **Vendor Relationship:** Vendors share details such as an address or repeatedly participate together.
5. **Unusual Participation:** A vendor's participation, withdrawals, losses or wins form an unusual pattern.

5. How does the engine work?

The engine does **not** simply say "good" or "bad." Instead, each detected pattern adds points to an **Investigation Priority Score**. More important signals add more points.

Example:

Repeated winner → +25 points

Price anomaly → +25 points

Very similar bids → +20 points

Vendor relationship → +20 points

Unusual participation → +10 points

Total: 87/100 → HIGH PRIORITY

6. The most important rule: explain every alert

If the system gives a score of 87, the investigator must be able to understand where that number came from. So every alert will contain the **signals, evidence and simple explanation**.

Example alert:

Vendor A won 9 of 10 comparable tenders.

The awarded price was 31% above the comparable median.

Vendor A and Vendor B share a registered address.

This does **not** mean corruption is proven. It means the case deserves human investigation.

7. Where Machine Learning fits

We can first build the engine using clear rules and statistics because they are easy to explain. After that, we can add Machine Learning to find unusual records that our fixed rules may miss.

Example: Isolation Forest can learn what normal procurement records generally look like and help identify records that look very different. Its result becomes **one additional signal** rather than the final decision.

8. Where the database fits

We can use **Supabase** as our database. After pandas cleans the uploaded file, we store the useful records in PostgreSQL through Supabase. This lets the frontend request tenders, vendors, alerts and investigation details later.

Basic database tables: Vendors • Tenders • Bids • Contracts • Payments • Alerts • Signals • Relationships

9. Complete backend flow

Step 1: User uploads CSV/Excel.

Step 2: FastAPI receives the file.

Step 3: pandas reads and cleans it.

Step 4: Clean records are stored in Supabase.

Step 5: Investigation Engine checks the records.

Step 6: Rules + statistics + optional ML generate signals.

Step 7: Scoring Engine combines signals into a 0–100 priority score.

Step 8: Backend saves the alert and its evidence.

Step 9: Frontend displays the alert, score, reasons and relationship graph.

10. Our technology stack

Purpose	Language / Technology	Library / Tool
Backend API	Python	FastAPI
CSV & Excel data	Python	pandas, openpyxl
Statistics	Python	NumPy, SciPy
Machine Learning	Python	scikit-learn
Relationships	Python	NetworkX
Database	SQL / PostgreSQL	Supabase
AI explanations	Python	LLM API (e.g. OpenAI)
Frontend	JavaScript / TypeScript	React / Next.js
Frontend graphs	JavaScript / TypeScript	Cytoscape.js / React Flow

Bottom line: Python handles the brain of the system. pandas handles the data, the Investigation Engine finds signals, the Scoring Engine prioritizes cases, Supabase stores everything, and FastAPI connects the backend to the frontend.